FastSET: Fast Sequential Retraining for Online Adaptation of Neural Receivers via Physics-Guided Decomposition

Shuo Wang, Facheng Hu, *Student Member, IEEE,*
Hongzi Zhu, *Senior Member, IEEE,* Yibo Pi, *Member, IEEE,* and Xudong Wang, *Fellow, IEEE*

***Abstract*—Neural receivers for wireless communication must adapt rapidly to changes in propagation conditions to ensure reliable performance. However, frequent online adaptation incurs significant latency and computational overhead, limiting real-time deployment. We propose FastSET, a fast sequential retraining strategy with a physics-guided estimator–demapper split based on the channel-estimation and demapping functions of conventional receivers. An offline model search balances bit error rate (BER) and adaptation time under a common parameter budget. FastSET first updates the estimator while freezing the demapper, then adapts the demapper if further updates are needed to reach the target BER. We evaluate FastSET in an orthogonal frequency-division multiplexing (OFDM) system using 3GPP tapped delay line (TDL) channel models and channels generated by NVIDIA Sionna ray tracing (RT) in four urban environments. FastSET reduces adaptation computation time by factors of 86.3 and 22.0 relative to end-to-end retraining on TDL-A-to-B and TDL-A-to-C transitions, respectively, with comparable post-adaptation BER. Code is available at https://github.com/pardonhu/FastSET.**

I. Introduction

Deep neural networks (DNNs) have been applied to wireless communication tasks including channel estimation [2]–[4] and channel coding [5], [6]. Neural receivers integrate learned components into the conventional receiver processing chain and can improve reception performance [7]–[10].

Compared with conventional receivers, neural receivers offer three advantages: 1) Reduced Pilot Overhead: Neural receivers can reduce the number of known reference symbols (pilots) needed for reception [11], [12], thereby enhancing spectral efficiency in high-frequency bands such as millimeter-wave and terahertz (THz) [13]. 2) Implicit Channel Characterization: Learning from received signals enables neural receivers to account for interference, non-linear hardware distortions, and quantization effects that are difficult to model analytically [14], [15]. 3) Optimized Complexity-Performance Trade-offs: Receiver architectures can balance computational cost and reception accuracy [16].

This work was supported by the National Natural Science Foundation of China under Grant No. 62371292. An earlier version of this paper was presented at the IEEE International Conference on Sensing, Communication, and Networking (SECON), 2026 [1].

Shuo Wang and Facheng Hu contributed equally and are co-first authors. The research was supervised by Xudong Wang.

Shuo Wang is with Alibaba Group.

Facheng Hu and Yibo Pi are with the Global College, Shanghai Jiao Tong University, Shanghai, China.

Hongzi Zhu is with the School of Computer Science, Shanghai Jiao Tong University, Shanghai, China.

Xudong Wang (corresponding author) is with the Internet of Things Thrust, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China. E-mail: wxudong@ieee.org.

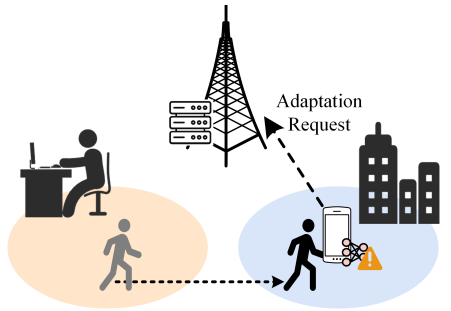


Fig. 1: A neural receiver pretrained in offline environment E_0 requires online adaptation after entering environment E_1.

However, a neural receiver pretrained in an offline environment E_0 can experience performance degradation when deployed in a different online environment E_1, as illustrated in Fig. 1. Online adaptation is then needed to restore reception accuracy. Conventional end-to-end retraining updates the entire model, incurring substantial computational overhead and delaying the recovery of reliable reception.

The core challenge is that online adaptation of neural receivers struggles to keep pace with rapid environmental changes. An effective adaptation scheme must satisfy three requirements: rapid convergence to time-varying channel conditions, recovery of the required reception accuracy under diverse scenarios (including environmental transitions, multipath effects, and Doppler shifts), and computational efficiency compatible with constrained base station resources. Prior works reduce adaptation overhead through lightweight receiver architectures [9] or meta-learning of suitable initializations [17]. Lightweight architectures can limit receiver capacity, while meta-learning retains the update costs of the chosen architecture. Reducing adaptation computation while preserving receiver capacity remains a challenge.

To reduce the time required to restore reliable reception after environmental changes, we propose **FastSET**, a **Fast SEquential reTraining** strategy for online adaptation of neural receivers. Accurate channel estimation plays a central role in reliable wireless reception by providing the channel information required for equalization and demapping. FastSET therefore splits the end-to-end (E2E) receiver into a neural estimator and a neural demapper. The estimator supplies an explicit channel estimate to the demapper. Updating the estimate allows reception to adapt to the target environment while the pretrained demapper weights remain fixed. The demapper is subsequently

adapted when further updates are needed to reach the target bit error rate (BER).

A second challenge is how to choose the estimator–demapper split and training schedule to reduce adaptation time while preserving reception accuracy. Under a fixed total parameter budget, increasing estimator capacity leaves fewer parameters for the demapper. The allocation affects the training cost of each module and the iterations needed to reach the target BER.

More specifically, an **Offline Model Search (OMS)** selects the estimator and demapper architectures under a common parameter budget. Both modules follow the backbone shape of the end-to-end receiver. Inspired by joint accuracy–latency objectives in neural architecture search [18], we define the **Performance–Efficiency Balance Score (PEBS)** and employ a grid search to select the split that balances BER and adaptation time. The selected split is used for subsequent online adaptation.

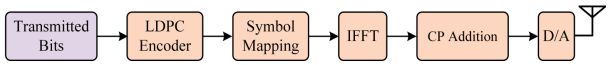
During **Online Sequential Adaptation (OSA)**, the FastSET protocol is triggered when the measured BER degrades beyond a threshold. The estimator is updated first with the demapper weights fixed. If further adaptation is needed, the estimator weights are fixed and the demapper is adapted using the updated channel estimates. Sequential training limits gradient computation and parameter updates to one module per iteration, reducing training cost. Adaptation ends when the complete receiver reaches the target BER.

We implement FastSET in an orthogonal frequency-division multiplexing (OFDM) system and conduct extensive simulations using NVIDIA Sionna with Third Generation Partnership Project (3GPP) tapped delay line (TDL) models and ray tracing (RT) in four urban environments: Paris, Munich, Florence, and San Francisco (SF). FastSET reduces adaptation computation time by factors of 86.3 on TDL-A-to-B (A2B) and 22.0 on TDL-A-to-C (A2C) compared with end-to-end retraining, with comparable post-adaptation BER.

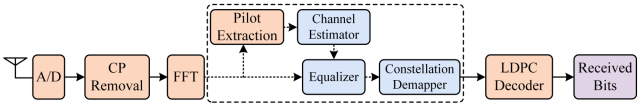
Our SECON paper [1] introduced estimator–demapper splitting, sequential retraining, and PEBS-based architecture selection. The journal version examines how the split and target BER affect the modules and iterations required for adaptation. Checking validation BER from the estimator stage reveals splits that meet the target with the demapper weights fixed, leading to the selection of 8–64 in place of the conference configuration, 5–16. The ablations relate the required module updates to the BER target and compare sequential and joint updates under matched initialization. The selected split is further evaluated across TDL transitions and four ray-traced cities, including cross-city adaptation.

The main contributions are:

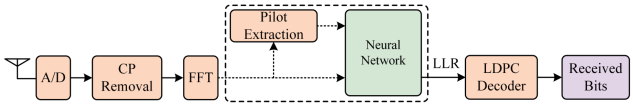
- A physics-guided estimator–demapper split following conventional receiver processing, with offline PEBS search to balance BER and adaptation time under a common parameter budget;
- A sequential adaptation protocol that updates the estimator first and the demapper only when needed to reach the target BER;
- Evaluations on 3GPP channel models and ray-traced urban scenarios that quantify adaptation-time savings and show when estimator adaptation alone reaches the BER target and when subsequent demapper adaptation improves BER.



(a) Transmitter structure for a single-input single-output (SISO) OFDM system



(b) Conventional receiver with discrete functional blocks



(c) Neural receiver with learned channel estimation, equalization, and demodulation

Fig. 2: Architecture comparison in a SISO-OFDM system. Blocks include low-density parity-check (LDPC) coding, inverse fast Fourier transform (IFFT), fast Fourier transform (FFT), cyclic prefix (CP) processing, digital-to-analog (D/A) conversion, and analog-to-digital (A/D) conversion.

II. Related Work

A. Model-Specific Neural Receiver Design

Lightweight neural receiver architectures reduce the computation required for online adaptation. Echo State Network (ESN)-based receivers [9] combine a fixed recurrent structure with a trainable linear output layer. The reservoir dynamics in ESNs encode temporal information without requiring back-propagation through time (BPTT), which significantly reduces computational overhead. Other lightweight architectures, such as shallow convolutional neural network (CNN) receivers or linear receivers with adaptive heads, similarly trade off model complexity for adaptation speed.

Limiting model size or trainable components reduces update cost but can restrict adaptation capacity. FastSET retains a comparable total parameter budget and splits the receiver according to the physical functions of channel estimation and demapping.

B. Neural Receiver Adaptation

Model-Agnostic Meta-Learning (MAML) [19] learns an initialization for adaptation with a few gradient steps. For neural receivers, Raviv et al. [17] develop predictive meta-learning from current and past channel observations, including modular training for model-based receivers. Adaptation time also depends on the computational cost of each update.

Uzlaner et al. [20] use drift detection to determine when and which receiver modules require retraining. FedPDA [21] reduces adaptation frequency through federated collaboration, using a main–side network and pruned-data aggregation coordinated among terminals and a base station.

Existing methods improve initialization or avoid unnecessary retraining, while each required adaptation can still incur

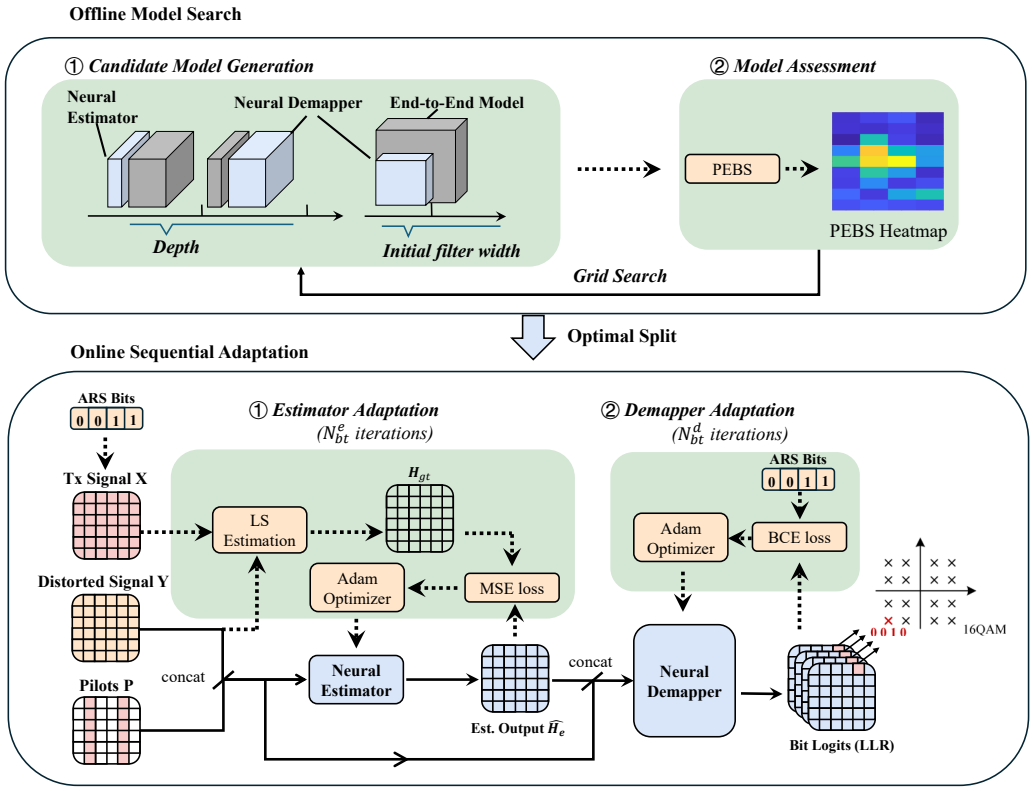


Fig. 3: Overview of the FastSET system, comprising two coordinated phases: Offline Model Search and Online Sequential Adaptation.

substantial computation. FastSET organizes adaptation around channel estimation and demapping, selecting the module sizes under a common parameter budget and updating the modules sequentially to reach the required BER.

III. Background

A. Conventional Non-Learning Receiver

Figure 2 shows the OFDM transmitter and receiver. After OFDM demodulation, the frequency-domain received signal is

$$Y[n,k] = H[n,k]X[n,k] + W[n,k], \tag{1}$$

where n and k index OFDM symbols and subcarriers, $X[n,k]$ is the transmitted symbol, $H[n,k]$ is the complex channel frequency response, and $W[n,k] \sim \mathcal{CN}(0,\sigma^2)$ is additive white Gaussian noise.

Channel Estimation: The conventional receiver in Fig. 2b estimates the channel from known pilot symbols. The Least Squares (LS) estimate is

$$H_{\mathrm{LS}}[n,k] = \frac{Y[n,k]}{P[n,k]}, \quad (n,k) \in \mathcal{P}, \tag{2}$$

where $H_{\mathrm{LS}}[n,k]$ denotes the LS channel estimate at a pilot position, $P[n,k]$ represents the known pilot symbol, and $\mathcal{P}$ is the set of pilot positions.

Equalization and Demodulation: The equalizer uses channel estimates to compensate for amplitude and phase distortion. The demapper converts equalized symbols into log-likelihood ratios (LLRs), expressing the relative likelihood of each coded bit being 0 or 1. The channel decoder uses the LLRs and coding constraints to recover information bits.

B. Neural Receiver Architecture

As depicted in Fig. 2c, the neural receiver replaces channel estimation, equalization, and demodulation with an end-to-end neural network. The network maps received signals to LLRs for subsequent channel decoding.

The input tensor combines the received signal, pilot information, and noise power:

$$\begin{aligned}\mathbf{Z} = \mathrm{Stack}\big(&\Re\{\mathbf{Y}\}, \Im\{\mathbf{Y}\}, \Re\{\mathbf{P}\}, \\ &\Im\{\mathbf{P}\}, N_0\big) \in \mathbb{R}^{N_t \times N_f \times 5},\end{aligned} \tag{3}$$

where $\Re\{\cdot\}$ and $\Im\{\cdot\}$ denote real and imaginary parts, $\mathbf{Y}$ and $\mathbf{P}$ are received-signal and pilot matrices, and N_t, N_f are the time and frequency dimensions. The noise-power input N_0 is supplied in logarithmic form and broadcast across the time–frequency grid in all neural receiver modules.

The end-to-end neural network implements the mapping:

$$\mathbf{L} = f_\theta(\mathbf{Z}), \tag{4}$$

where θ denotes network parameters and $\mathbf{L}$ contains output LLRs. Training minimizes the binary cross-entropy (BCE) loss:

$$\mathcal{L}_{\mathrm{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[Q_i \log(\sigma(L_i)) + (1-Q_i)\log(1-\sigma(L_i))\right]. \tag{5}$$

Here, $Q_i \in \{0,1\}$ and L_i are the transmitted coded bit and predicted LLR, σ is the sigmoid function, and N is the number of coded bits.

C. Online Adaptation Challenges

Neural receivers adapt to changing environments by retraining on newly acquired labeled examples. The total adaptation latency is

$$\Delta t_A = \Delta t_R + \Delta t_T + \Delta t_V, \tag{6}$$

where Δt_R is labeled-data acquisition time, Δt_T is adaptation computation time, and Δt_V is the cumulative time spent checking BER. The computation term includes data reading, processing, and training in a loaded, warmed-up receiver.

E2E adaptation updates all model parameters, increasing computation when propagation conditions change. FastSET reduces retraining overhead through sequential estimator–demapper updates.

IV. System Design

FastSET uses the channel estimate as an explicit intermediate representation between the estimator and demapper (Fig. 3). The estimator adds a learned correction to an LS estimate. The demapper combines the resulting estimate with received signals, pilots, and noise power to produce LLRs. Updating the estimator changes the demapper input while preserving the pretrained demapper weights. OMS selects the module architectures under a common parameter budget; OSA updates the modules sequentially until the target BER is reached or the adaptation budget is exhausted.

A. Offline Model Search (OMS)

OMS balances BER and adaptation time under the parameter budget of the CNN-based E2E receiver.

1) Candidate Model Generation: Let $(d_{\mathrm{E2E}}, w_{\mathrm{E2E}})$ denote the E2E residual-block count and initial filter width. OMS searches estimator depth d_e and initial filter width w_e within:

$$d_e \in [d_{\min}, d_{\max}], \quad w_e \in [w_{\min}, w_{\max}], \tag{7}$$

The estimator is shallower than E2E, with no greater initial filter width; the two modules share the E2E parameter budget.

Both modules follow the spindle-shaped E2E residual backbone, with filter width increasing toward the middle and decreasing toward the output. A split d_e–w_e specifies estimator depth and initial filter width; 8–64 denotes eight residual blocks with initial filter width 64. The remaining parameter budget determines the demapper architecture.

An estimator with $\mathcal{P}_e(d_e, w_e)$ parameters leaves the demapper a budget of:

$$\mathcal{P}_d = \mathcal{P}_{\mathrm{E2E}} - \mathcal{P}_e(d_e, w_e), \tag{8}$$

where $\mathcal{P}_{\mathrm{E2E}}$ denotes the total number of trainable parameters of the original E2E receiver.

The demapper depth d_d is proportional to the remaining budget:

$$d_d = \left\lfloor \frac{\mathcal{P}_d}{\mathcal{P}_{\mathrm{E2E}}} \cdot d_{\mathrm{E2E}} \right\rfloor. \tag{9}$$

With d_d fixed, the demapper initial filter width w_d is selected by bisection to approach the remaining budget from below:

$$\mathcal{P}_e(d_e, w_e) + \mathcal{P}_d(d_d, w_d) \approx \mathcal{P}_{\mathrm{E2E}}, \tag{10}$$

The combined parameter count does not exceed the E2E budget.

2) Model Assessment: For a common comparison, E2E is adapted from source-pretrained weights until validation BER first reaches the target. The adaptation time T_{E2E} is recorded, and $\mathrm{BER}_{\mathrm{E2E}}$ is measured on separate evaluation samples with the selected weights fixed. All candidates use the same reference.

Each candidate completes a fixed adaptation budget without early stopping, with validation after every iteration. Let $\mathcal{K}_c$ index the models saved during estimator and demapper adaptation that meet the validation BER target. For each $k \in \mathcal{K}_c$, $\mathrm{BER}_{c,k}$ is measured on the same evaluation samples as E2E. Cumulative time $T_{c,k}$ includes input processing and training, excluding BER evaluation.

Following the joint consideration of accuracy and execution cost in neural architecture search [18], candidate c is scored using the highest PEBS among the models indexed by $\mathcal{K}_c$:

$$\mathrm{PEBS}(c;\alpha) = \max_{k \in \mathcal{K}_c} \left[\alpha \frac{\mathrm{BER}_{\mathrm{E2E}}}{\mathrm{BER}_{c,k}} + (1-\alpha) \frac{T_{\mathrm{E2E}}}{T_{c,k}} \right]. \tag{11}$$

The two ratios measure BER reduction and adaptation speedup relative to E2E, weighted by $\alpha \in [0,1]$ and $1-\alpha$, respectively. The E2E reference has a score of one; a higher PEBS indicates a larger weighted gain at the chosen α. Further training can improve BER while increasing adaptation time.

OMS selects the highest-scoring architecture from the fixed-budget comparison. For each subsequent online adaptation, the selected architecture is initialized with the corresponding source-pretrained weights, and OSA stops when validation BER first meets the target.

B. Online Sequential Adaptation (OSA)

Once a split is selected, OSA updates the estimator first, followed by the demapper if needed, with the other module frozen (Fig. 3). Simulated channel responses supervise the estimator; known transmitted coded bits supervise the demapper.

1) Estimator Adaptation: The estimator is updated using the channel-estimation loss for at most N_e^{bt} iterations. After each iteration, BER is evaluated after channel decoding, and adaptation ends if the target is reached. The updated estimates change the demapper input while the demapper weights remain fixed.

The estimator input is:

$$\begin{aligned} \mathbf{Z}_e = \mathrm{Stack}\big(&\Re\{\mathbf{Y}\}, \Im\{\mathbf{Y}\}, \\ &\Re\{\mathbf{P}\}, \Im\{\mathbf{P}\}, N_0\big), \end{aligned} \tag{12}$$

The pilot-position LS estimate is:

$$H_{\mathrm{LS}}[n,k] = \frac{Y[n,k]}{P[n,k]}, \quad (n,k) \in \mathcal{P}, \tag{13}$$

The estimates are linearly interpolated over the resource grid and added to the predicted channel correction:

$$\hat{\mathbf{H}}_{e,\mathrm{RI}} = \mathbf{H}_{\mathrm{LS,RI}} + r_{\theta_e}(\mathbf{Z}_e). \tag{14}$$

RI denotes real and imaginary components. The network output $r_{\theta_e}(\mathbf{Z}_e)$ is the predicted channel correction.

TABLE I: Sionna TDL simulation and dataset parameters.

Parameter	Value
Link / carrier	SISO OFDM / 4 GHz
Subcarriers / spacing	264 / 15 kHz
OFDM symbols / CP	14 / 16 samples
Pilots	Kronecker; symbols 2, 11
Guards / DC nulling	None / disabled
Modulation / coding	16-QAM / 5G LDPC
Info./coded bits per frame	8,142 / 12,672
TDL-A source	50 ns; 0–1.5 m/s
TDL-B source (B2C)	150 ns; 0–3 m/s
TDL-B target (A2B)	500 ns; 0–5 m/s
TDL-C target (A2C, B2C)	1,000 ns; 0–10 m/s
Pretraining SNR	4–18 dB; 1 dB steps
Pretraining frames/SNR	6,400
Online batch size	32 frames
Epochs (est./dem.; E2E)	5/10; 10
Adam learning rate / epsilon	10^{-3} / 10^{-7}
Test frames/SNR	3,200
Test information bits/SNR	26,054,400
Validation BER target	BER $< 2 \times 10^{-4}$ at 12 dB

Pilot indices are zero-based; DC denotes direct current.

The training objective minimizes the mean squared error (MSE) between the channel estimate and the true simulated channel response:

$$\theta_e^* = \arg\min_{\theta_e} \mathbb{E}\Big[\mathrm{mean}\big((\hat{\mathbf{H}}_{e,\mathrm{RI}} - \mathbf{H}_{\mathrm{true,RI}})^2\big)\Big]. \tag{15}$$

True-channel labels are used for supervised training and channel-error evaluation; inference uses received samples, pilots, and noise power.

2) Demapper Adaptation: If estimator adaptation leaves the BER target unmet, the demapper is updated for at most N_d^{bt} iterations, with the estimator fixed and early stopping at the target. The demapper input is:

$$\begin{aligned}\mathbf{I}_d = \mathrm{Concat}\big(&\Re\{\mathbf{Y}\}, \Im\{\mathbf{Y}\}, \Re\{\mathbf{P}\},\\ &\Im\{\mathbf{P}\}, \hat{\mathbf{H}}_{e,\mathrm{RI}}, N_0\big),\end{aligned} \tag{16}$$

where $\hat{\mathbf{H}}_{e,\mathrm{RI}}$ contains the real and imaginary parts of the channel estimate from the frozen estimator in (14). The frozen estimator computes a channel estimate for each received sample using the same weights throughout demapper adaptation. The resulting estimates vary with the received signals and pilots.

The demapper outputs coded-bit LLRs for the LDPC decoder to recover information bits and is trained using the BCE loss.

The estimator is adapted to the channel response through MSE, while the demapper is adapted to coded bits through BCE. During estimator adaptation, gradients are computed from the channel-estimation loss without backpropagating through the demapper. The updated channel estimates are then evaluated by the complete receiver using post-decoding BER. Adaptation ends when the fixed demapper and decoder recover the target BER. If the estimator budget is exhausted before the target is reached, the updated estimates are used for subsequent demapper adaptation.

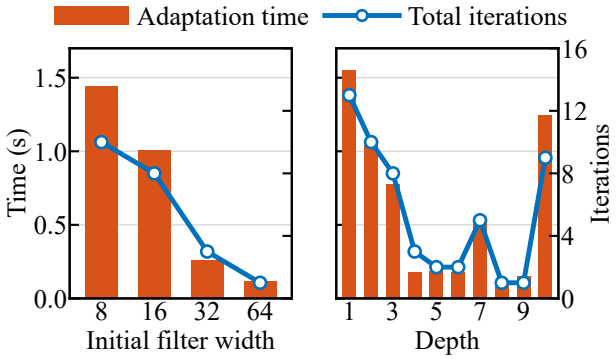


Fig. 4: Estimator initial filter width and depth on A2B. Left: varying initial filter width at depth eight. Right: varying depth at initial filter width 64. Bars show adaptation time; curves show total iterations to reach validation BER $< 2 \times 10^{-4}$ at 12 dB. The outer axes apply to both panels.

V. Experimental Evaluation

A. Experimental Setup

The experiments compare the computation required to restore a target BER after a change in propagation conditions. FastSET and E2E share the parameter budget, adaptation data, and optimizer settings. Source-domain BER is comparable in the main TDL comparison (Supplementary Section II). Standardized TDL profiles and Sionna RT urban scenes provide the source and target environments. TDL-A, TDL-B, and TDL-C differ in multipath characteristics. A2B denotes pretraining on TDL-A followed by adaptation to TDL-B; A2C and B2C use the same notation. Delay spread describes the dispersion of multipath arrival times. Table I lists the link and training parameters, including 16-ary quadrature amplitude modulation (QAM) and fifth-generation (5G) LDPC coding. Each pretraining epoch uses 96,000 frames over 4–18 dB, and each online iteration processes 32 frames. Signal-to-noise ratio (SNR) is expressed as E_b/N_0, the ratio of energy per information bit to noise power spectral density.

Adaptation samples are generated independently of evaluation samples. After each online iteration, the model is saved and post-LDPC BER is measured on $\mathcal{D}_{\mathrm{val},1}$ to locate the first iteration below 2×10^{-4} at 12 dB. Adaptation computation time Δt_T in Equation (6), abbreviated as adaptation time, is measured by replaying the same updates and adaptation data with model saving and BER evaluation disabled. For the main TDL curves, $\mathcal{D}_{\mathrm{val},1}$ is reused as $\mathcal{D}_{\mathrm{test}}$; separate samples $\mathcal{D}_{\mathrm{val},2}$ are used for PEBS ranking. Final A2B and A2C models are additionally tested at 12 dB on newly generated $\mathcal{D}_{\mathrm{ind}}$ (Supplementary Section VII). BER–SNR curves use logarithmic BER axes.

E2E full-model adaptation is the main baseline for adaptation time and BER. A CNN estimator with conventional demapping (CNN est.) and LS-based reception compare alternative channel-estimation methods. Perfect channel state information (CSI) reception uses the true channel response as a reference. Supplementary Section I reports the batch-size comparison.

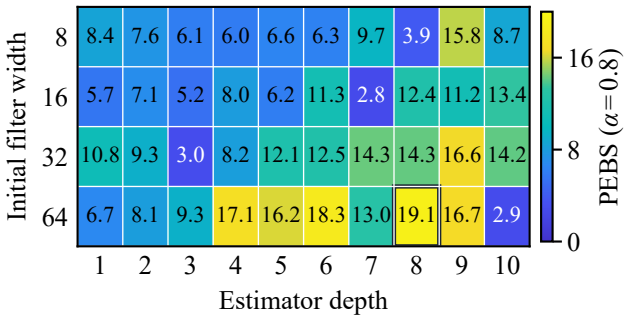


Fig. 5: PEBS of 40 candidate splits on A2B at $\alpha = 0.8$. Each cell reports the maximum PEBS among models meeting the validation BER target within 25 adaptation iterations. Rows vary estimator initial filter width; columns vary estimator depth.

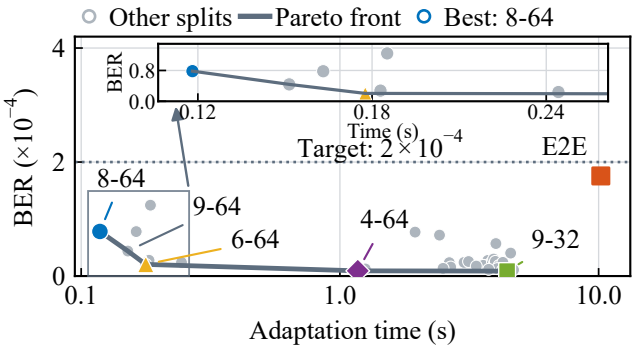


Fig. 6: BER–time trade-off at each split's highest-PEBS model on A2B ($\alpha = 0.8$). Time is logarithmic in the main panel and linear in the inset; both BER axes are linear, in units of 10^{-4}.

FastSET is evaluated on a server with an NVIDIA GeForce RTX 3090 Ti GPU, an AMD Ryzen 7 9800X3D CPU, and a 1 TB KIOXIA EXCERIA PLUS G4 SSD, using TensorFlow 2.15.1 and NVIDIA Sionna 0.19.2 [22].

B. Hyperparameter Selection

1) Estimator Initial Filter Width and Depth: Estimator initial filter width and depth are varied to examine how capacity allocation affects adaptation on A2B. The reference E2E model has 11 residual blocks, initial filter width 64, and 661,188 parameters. The search covers estimator depths 1–10 and initial filter widths 8/16/32/64, with the maximum width matching E2E. The remaining parameter budget determines the demapper. Figure 4 compares widths at depth eight and depths at initial filter width 64, using source-pretrained weights and the same stopping rule. At depth eight, increasing initial filter width reduces both adaptation time and the required iterations: widths 8 and 64 take 1.443 and 0.118 s, respectively. The two widest estimators reach the target before demapper adaptation begins. Greater estimator capacity therefore enables the channel-estimation stage to recover the target BER with fewer iterations in the depth-eight comparison.

The depth comparison shows that allocating more layers to the estimator does not consistently accelerate adaptation. At initial filter width 64, depths eight and nine reach the target after one estimator iteration, whereas depth ten also requires demapper adaptation (Figure 4, right). The difference motivates searching depth and initial filter width together under the common parameter budget.

2) Split Selection with PEBS: To select a split that balances reception performance and adaptation time, all 40 candidates are assessed offline on A2B. Each completes five estimator and 20 demapper iterations; models meeting the validation target on $\mathcal{D}_{\mathrm{val},1}$ are evaluated on $\mathcal{D}_{\mathrm{val},2}$ for PEBS ranking at $\alpha = 0.8$. The full search trajectory permits comparisons across training stages; subsequent online adaptation stops at the first validation success. E2E first meets the target at iteration 50, providing a reference BER of 1.76×10^{-4} and time of 10.197 s.

Figure 5 ranks the 8–64 split first with PEBS 19.06. The split reaches the target after one estimator iteration and also meets the target on $\mathcal{D}_{\mathrm{val},2}$. The weighted BER and time terms are 1.80 and 17.26, so the score mainly reflects rapid adaptation. By comparison, the 7–64 split attains a lower BER after further training but takes 1.254 s and receives a lower score. The selected split uses 630,870 parameters, below E2E's 661,188.

Figure 6 clarifies the performance–time trade-off using each split's highest-PEBS model. Time is logarithmic in the main panel and linear in the inset; both BER axes are linear. The 8–64 split occupies the fastest end of the Pareto frontier, reaching BER 7.82×10^{-5} in 118.14 ms. The 4–64 split reduces BER to 9.13×10^{-6} at a cost of 1.172 s, and the 9–32 split extends the trade-off toward still lower BER and longer adaptation. Thus, 8–64 favors rapid recovery, while other frontier splits favor further BER reduction.

To examine whether the selection depends on the PEBS weight, each split's maximum score is recomputed across α (Figure 7). The 8–64 split ranks first at $\alpha = 0.2, 0.4, 0.6, 0.8$. As the BER term receives more weight, the preferred split changes to 4–64 and then 9–32. The ranking follows the trade-off in Figure 6: lower final BER becomes preferable to faster adaptation. FastSET uses 8–64 at $\alpha = 0.8$ in subsequent CNN-backbone experiments. Including models saved during estimator adaptation allows the search to recognize recovery with fixed demapper weights; the main experiments then test the selected architecture beyond A2B.

3) Learning Rate: The learning-rate experiment examines how update size affects recovery speed and subsequent BER. With 8–64 fixed, five rates, 10^{-2}, 3×10^{-3}, 10^{-3}, 3×10^{-4}, and 10^{-4}, are compared on A2B at 12 dB using a batch size of 32, five estimator iterations, and 20 demapper iterations (Figure 8). Training continues beyond the BER target to expose later behavior. A rate of 10^{-3} reaches the target after one estimator iteration, compared with three estimator iterations at 3×10^{-4}. The smaller rate produces less fluctuation and lower BER by iteration 25. 10^{-3} is selected for faster recovery to the target BER.

C. Main Performance

The main comparison tests whether the selected split preserves reception performance while accelerating adaptation, including on a target outside the split search. Receivers pretrained on TDL-A with similar source-domain BER are

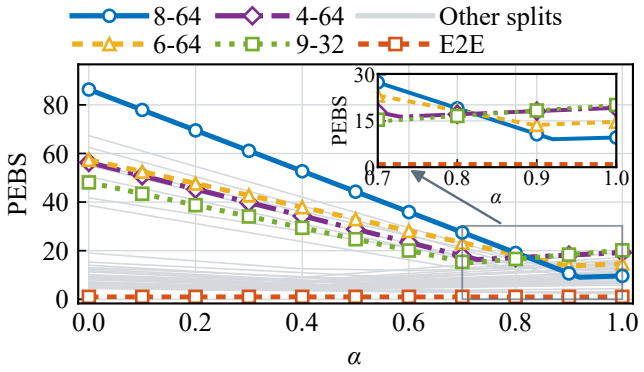


Fig. 7: Sensitivity of split selection to α. Each curve reports a split's maximum PEBS among models meeting the validation BER target within 25 adaptation iterations. The inset shows where the highest-scoring split changes.

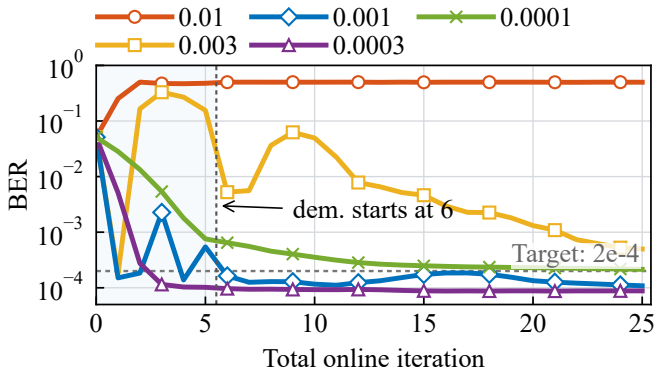


Fig. 8: Learning-rate sensitivity on A2B at 12 dB. Iteration 0 denotes the pretrained 8–64 receiver. The BER axis is logarithmic; the horizontal line marks 2×10^{-4}.

adapted to the B and C targets in Table I. After reaching the validation target at 12 dB, weights are fixed across the SNR evaluation. Figure 9 shows comparable post-adaptation BER for FastSET and E2E on both transitions, with both improving on LS-based reception. The A2C result supports using the A2B-selected architecture without another split search.

Figure 10 shows how quickly the receivers recover the validation BER target. The time axis is linear up to 0.1 s and logarithmic thereafter, retaining the pretrained result at time zero. On A2B, FastSET requires one estimator iteration and 118.14 ms, compared with 50 iterations and 10.197 s for E2E (86.3× speedup). Updating the channel estimate is sufficient to restore reception with the demapper fixed. On A2C, five estimator iterations are followed by one demapper iteration under the default schedule, giving a 22.0× speedup. Fewer iterations and lower per-iteration training cost contribute to the speedups (Section V-F). Equal-time and training-set-size comparisons are provided in Supplementary Sections V and VI.

Independent evaluation on $\mathcal{D}_{\text{ind}}$ checks performance after the stopping models have been fixed (Supplementary Section VII). FastSET and E2E achieve BERs of 3.02×10^{-4} and 4.15×10^{-4} on A2B, and 6.78×10^{-5} and 4.08×10^{-4} on A2C. FastSET retains lower test BER on both transitions; only FastSET on A2C remains below 2×10^{-4}.

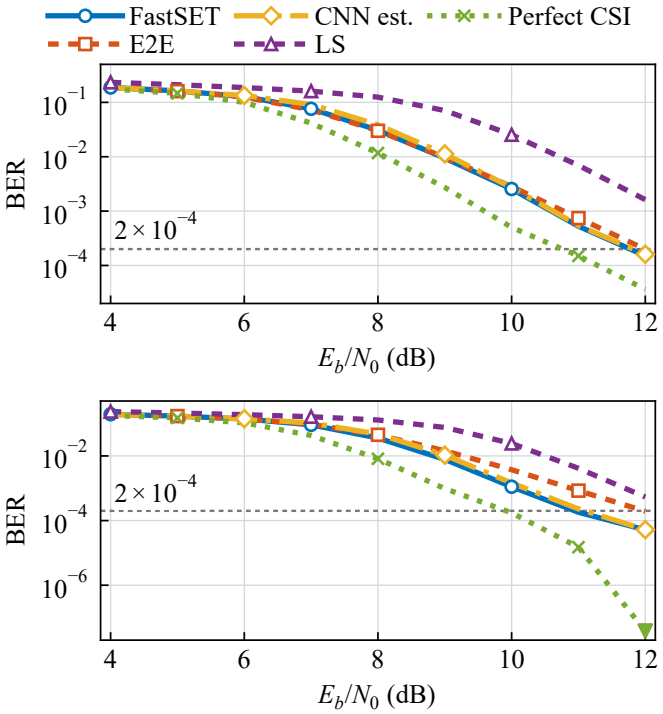


Fig. 9: Post-adaptation BER on A2B (top) and A2C (bottom), with fixed weights across SNRs. Downward triangles mark zero-error observations, displayed at $1/26{,}054{,}400$.

D. Ablation of the Adaptation Strategy

The ablations examine whether estimator adaptation alone restores the target BER, how sequential updates compare with joint updates, and how the estimator budget affects the achievable BER. Two FastSET schedules are compared with Est. only, Dem. only, two Joint variants, and E2E on A2B and A2C. FastSET (Est. 1) uses one estimator iteration before demapper adaptation; FastSET (Est. 5) uses five estimator iterations. The time to reach the BER target is recorded even when the target is reached before all scheduled estimator iterations are completed. The single-module controls share FastSET's pretrained weights and freeze the other module. Joint (SP) uses the same architecture and pretrained model as FastSET to compare sequential and joint online updates under matched initialization. Joint (JP) also uses joint pretraining. Both variants update both modules online with equally weighted MSE and BCE losses. Supplementary Section III details the matched-initialization comparison.

Three runs use different online batch orders and the same $\mathcal{D}_{\text{test}}$; each method starts from the same source-pretrained weights across runs. A total of 50 iterations on A2B and 84 iterations on A2C are recorded, with training continued beyond the BER target. For FastSET, the total includes both estimator and demapper iterations. Figures 11 and 12 compare the adaptation strategies using logarithmic BER and time axes. Curve times are averaged at matching iterations; the time to reach each BER target is computed separately for each run. Curve starting times reflect the first iteration: estimator updates are faster than Dem. only, Joint, and E2E updates.

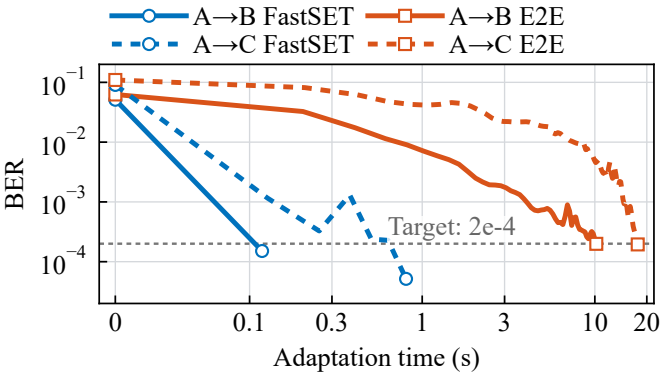


Fig. 10: FastSET and E2E adaptation on A2B and A2C at 12 dB. The horizontal line marks the BER target 2×10^{-4}. The time axis is linear up to 0.1 s and logarithmic thereafter.

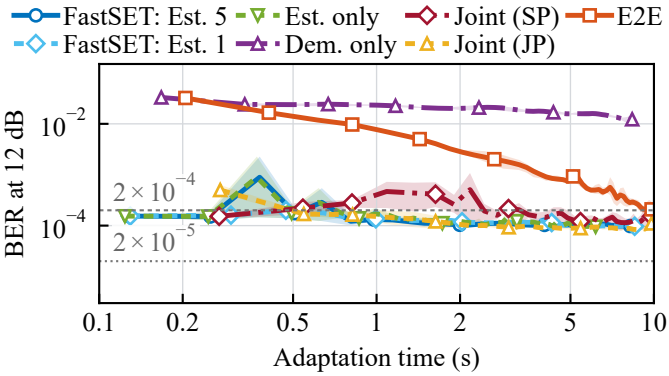


Fig. 11: Adaptation strategies on A2B at 12 dB. Curves show mean BER with minimum–maximum bands across three runs. The first 10 s of the 50-iteration experiment are shown.

A2B: Figure 11 shows that estimator adaptation is sufficient to recover the target BER. Both FastSET schedules and Est. only reach 2×10^{-4} after one estimator iteration in all three runs, while Dem. only remains above the target throughout the budget. The channel estimate can therefore be adapted while the pretrained demapper is retained. FastSET and Joint (SP) both need one iteration, but FastSET (Est. 5) takes 0.128 s on average, compared with 0.271 s for Joint (SP). The matched-initialization comparison shows the cost saved by updating only the estimator. E2E requires 44–50 iterations. No method reaches 2×10^{-5} within the budget.

A2C: Figure 12 shows that the preferred estimator budget depends on the target BER. FastSET (Est. 1) reaches 2×10^{-4} in 2–3 total iterations, sooner than the 3–6 iterations required by FastSET (Est. 5). At 2×10^{-5}, Est. 5 succeeds in all three runs, while Est. 1 remains above the target within 84 iterations. Est. only also remains above the stricter target, and Dem. only reaches neither target. Rapid recovery to the looser target favors a short estimator stage; the stricter target benefits from further channel-estimation refinement followed by demapper adaptation.

With matched initialization, sequential online updates have lower mean adaptation time than Joint (SP) on both transitions. On A2C, FastSET (Est. 5) averages 0.553 s to reach 2×10^{-4},

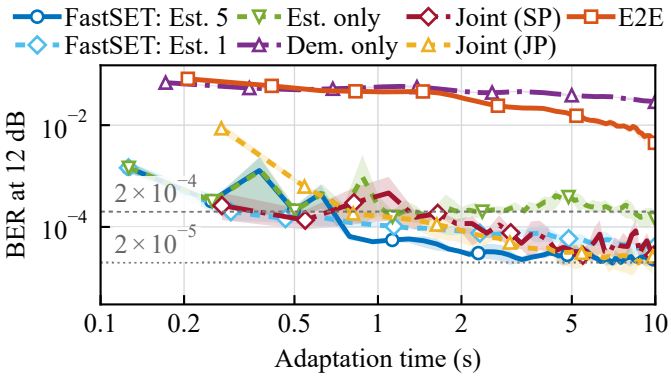


Fig. 12: Adaptation strategies on A2C at 12 dB. Curves show mean BER with minimum–maximum bands across three runs. The first 10 s of the 84-iteration experiment are shown. Horizontal lines mark BER targets 2×10^{-4} and 2×10^{-5}.

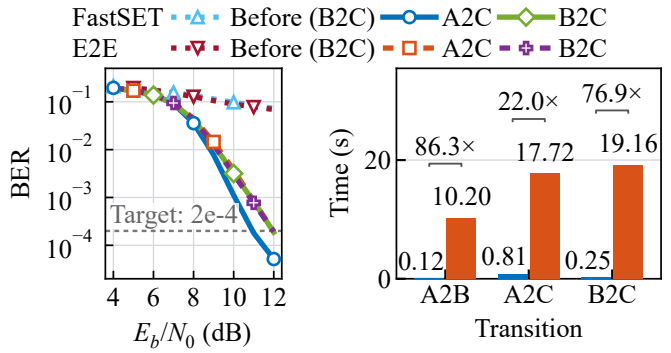


Fig. 13: Adaptation across TDL channel transitions. Left: A2C/B2C BER after adaptation, with pretrained curves shown for B2C. Right: adaptation time for FastSET (blue) and E2E (orange); labels give the E2E/FastSET time ratio. Adaptation ends when validation BER falls below 2×10^{-4} at 12 dB.

compared with 0.635 s for Joint (SP). The comparison isolates the online update strategy; the E2E comparison evaluates the complete design, including split architecture, supervision, pretraining, and online updates. Joint (JP) also changes pretraining and reaches the stricter A2C target faster in one run. Across three runs, FastSET has lower mean time to that target, while continued Joint (JP) training yields lower final BER.

E. Impact of Channels, Delays, and Speeds

To test whether the selected split also works with a different pretraining environment, the source is changed from TDL-A to TDL-B while the TDL-C target is retained. B2C pretraining uses a delay spread of 150 ns and speeds of 0–3 m/s. Figure 13 shows that the fixed 8–64 split retains the adaptation-time advantage: FastSET reaches the target after two estimator iterations, compared with 92 E2E iterations (76.90× speedup). As in A2B, updated channel estimates allow the pretrained demapper to recover the target BER.

The delay experiment varies the spread of multipath arrival times while fixing the TDL-A pretrained models and target speed range. In Figure 14, increasing TDL-B delay spread from 600 to 1200 ns raises adaptation time for both receivers.

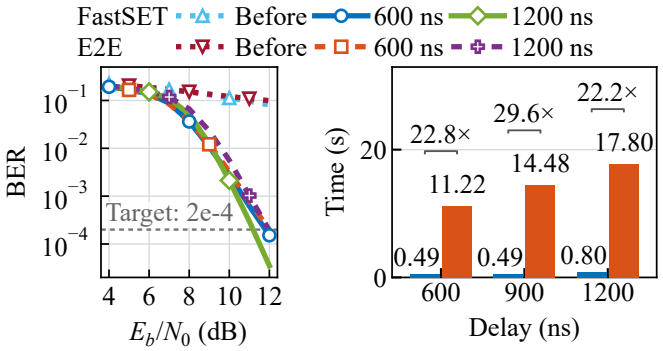


Fig. 14: Delay-spread sensitivity on TDL-B with fixed TDL-A pretrained receivers. Left: BER after adaptation at 600 and 1200 ns, with pretrained curves shown at 600 ns. Right: adaptation time at 600, 900, and 1200 ns for FastSET (blue) and E2E (orange), with the E2E/FastSET time ratio.

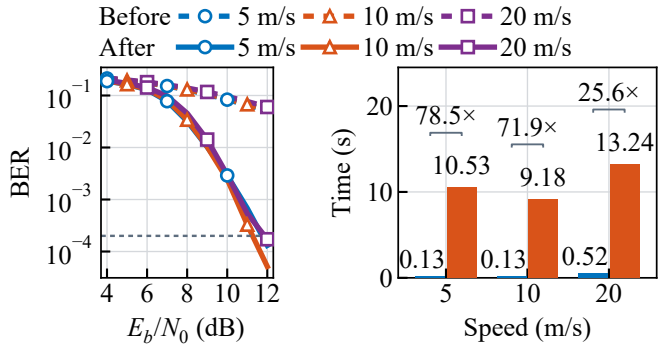


Fig. 15: Speed sensitivity on TDL-B at a delay spread of 500 ns. Left: FastSET BER before and after adaptation. Right: time to reach validation BER $< 2 \times 10^{-4}$ at 12 dB for FastSET (blue) and E2E (orange); labels give the time ratio.

FastSET remains approximately 22× faster at both endpoints. The increased delay spread changes the channel responses presented to the receiver; the measured time increase shows that more adaptation computation is required, while the split retains a similar relative advantage.

The speed experiment evaluates sensitivity to Doppler changes at target speeds of 5, 10, and 20 m/s, with the TDL-B delay spread fixed at 500 ns and the TDL-A pretrained models unchanged. Figure 15 shows nearly overlapping post-adaptation FastSET BER curves around 10 dB. The highest speed requires the longest adaptation time and gives the smallest speedup, 25.6×. Thus, similar final reception performance is recovered across the tested speeds, with greater computational effort at 20 m/s.

F. Breakdown of Adaptation Computation Time

The timing breakdown examines whether the acceleration comes from fewer iterations, cheaper updates, or input processing. Table II separates file reading, deserialization/checking, tensor preparation/transfer, and training. Each sequence is replayed three times, and the run with median total time is reported. Training accounts for 96.6–98.2% of the total.

TABLE II: Adaptation iterations and time components on A2B and A2C. FastSET iteration counts are reported as estimator/demapper; E2E counts refer to full-model iterations. All time components are in milliseconds.

	A2B		A2C	
	FastSET	E2E	FastSET	E2E
Retraining iterations	1/0	50	5/1	84
File reading	1.39	32.43	4.95	109.76
Deserialization/checks	1.11	53.17	6.23	91.73
Tensor preparation/transfer	1.92	103.46	11.67	173.67
Training	124.18	10509.60	795.79	17651.02
Total (ms)	128.60	10698.65	818.64	18026.18

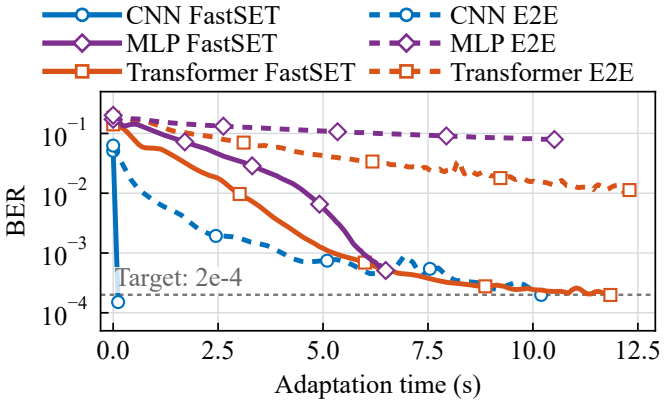


Fig. 16: FastSET and E2E adaptation with CNN, Transformer, and MLP backbones. Colors and markers identify the backbone; solid and dashed lines distinguish FastSET and E2E.

On A2B, a FastSET estimator iteration takes 124.18 ms of training, compared with an average 210.19 ms for E2E. Combined with the reduction from 50 iterations to one, the results show that fewer iterations contribute most of the A2B speedup, with an additional saving from updating one module. Input overhead is small, so the advantage remains when data handling is included. The component measurements use separate replays with additional timing instrumentation, accounting for differences from the main results.

G. Impact of Receiver Backbones

To examine whether the benefit depends on the CNN backbone, estimator–demapper splitting is also applied to Transformer and multilayer perceptron (MLP) receivers. The splits follow the estimator–demapper ratio of 8–64 (Figure 16). The CNN adapts fastest, while splitting also improves the Transformer and MLP relative to the corresponding E2E models. For example, the split Transformer reaches the target after five estimator and 130 demapper iterations; E2E remains at BER 1.13×10^{-2} after 200 iterations. Both MLP variants remain above the target, with lower final BER for the split model. Final BER is reported for methods that exhaust the budget without reaching the target.

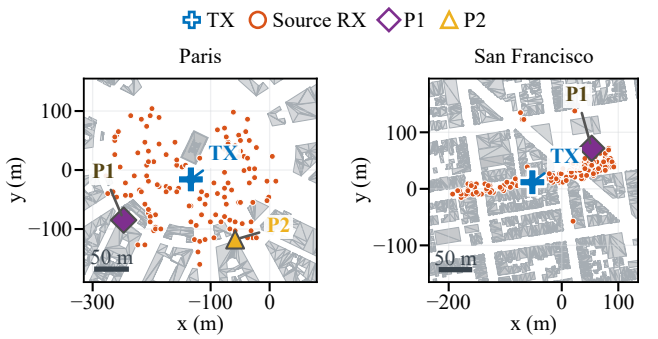


Fig. 17: Paris and San Francisco RT layouts. TX denotes the transmitter; orange dots mark source receivers. P1 and P2 mark target receiver positions. Coordinates are in meters.

TABLE III: RT target positions and RMS delay spreads computed from ray-traced path powers and delays. Coordinates use each scene's coordinate system.

Target	RX (x, y, z) (m)	RMS delay (ns)
Paris P1	$(-247.45, -84.89, 1.50)$	1189.77
Paris P2	$(-58.45, -117.89, 1.50)$	1044.56
Munich P1	$(87.60, 29.23, 1.50)$	864.93
Florence P1	$(-60.75, -1.79, 1.50)$	573.14
SF P1	$(52.97, 71.72, 12.13)$	340.14

H. Evaluation with NVIDIA Sionna RT

The RT experiments test the A2B-selected architecture on propagation paths generated from three-dimensional (3D) urban geometry by NVIDIA Sionna RT [23]. Figure 17 marks transmitters, source receivers, and adaptation targets P1 and P2 in Paris and San Francisco. FastSET and E2E are pretrained separately for Paris, Munich, Florence, and San Francisco. Each source dataset contains 128 line-of-sight (LOS) links at horizontal distances of 25–150 m; median root-mean-square (RMS) delay spreads are 121.20, 52.72, 73.58, and 46.13 ns, respectively. Table III lists fixed non-line-of-sight (NLOS) targets. Ray tracing supplies path gains, delays, and directions. Responses are normalized to unit mean power with Doppler shifts for 1 m/s. Targets add independent quadrature phase-shift keying (QPSK) interference on 33 subcarriers (495 kHz) at a full-band signal-to-interference ratio (SIR) of 15 dB; sources have no added interference.

RT adaptation cycles through SNRs of 4–18 dB, with stopping determined by 256 validation frames at 12 dB. Testing uses 3,200 independent signal, noise, and interference realizations per SNR at each target position. Figure 18 compares reception before and after adaptation. Both methods recover the target BER in all four cities, including Paris and Munich where pretrained BER is high for both receivers. Around 10–12 dB, FastSET has lower BER in Paris and Munich, while E2E has lower BER in SF and Florence. The fixed split therefore supports recovery across the urban targets, with the relative final BER varying by city.

Figure 19 shows that the size of the time saving depends on the adaptation work required by E2E. At Paris P1, FastSET uses seven total iterations compared with 22 for E2E, reducing time from 4.618 to 0.975 s ($4.74\times$ speedup). At the Munich,

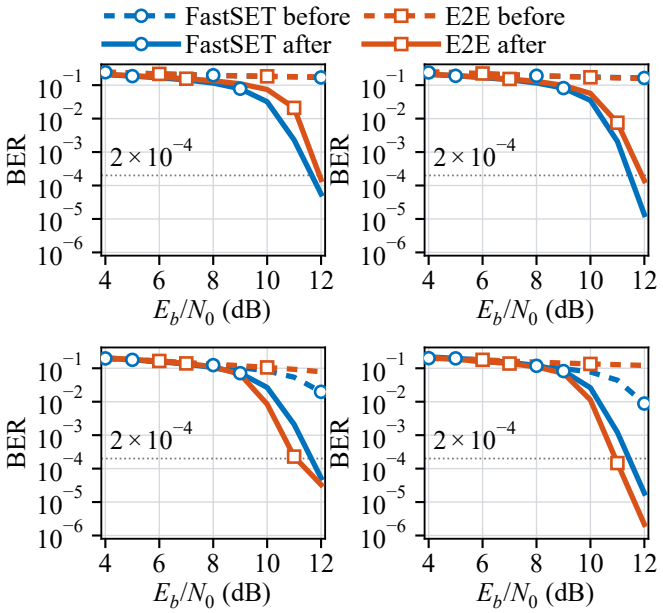


Fig. 18: BER at P1 before and after adaptation. Top: Paris and Munich; bottom: San Francisco and Florence.

Florence, and San Francisco targets, either receiver requires only six to eight iterations, leaving less opportunity to reduce the iteration count. FastSET still benefits from lower average iteration cost: at San Francisco P1, seven module updates take less time than six E2E updates ($1.29\times$ speedup). The larger Paris gain combines fewer iterations with cheaper updates; the smaller gains in the other cities follow from the short E2E adaptation sequences.

To examine variation within a city, Paris P1 and P2 are compared using the same pretrained receivers and interference settings (Figure 20). Both receivers recover low BER at both positions, while P2 requires more FastSET adaptation: after five estimator iterations, the demapper needs eight iterations at P2 compared with two at P1. FastSET remains $2.45\times$ faster than E2E at P2 (Figure 19). The additional demapper work shows that the same estimator budget leaves different amounts of adaptation to the demapper across target environments.

Cross-city adaptation tests transfer from a pretrained city to a target in another city, with the 8–64 split retained. Figure 21 compares Paris-to-San-Francisco and San-Francisco-to-Paris adaptation. FastSET is faster in both directions, with a larger speedup toward Paris ($4.45\times$) than toward SF ($1.25\times$). The adapted BER curves nearly coincide at 12 dB for the Paris target. Paris P1 has greater RMS delay spread and higher pre-adaptation BER than SF P1, consistent with the longer E2E adaptation sequence.

To isolate delay spread from position and interference, the excess path delays of Paris P1 are rescaled while path gains, receiver position, speed, and interference remain fixed (Figure 22). From 600 to 1200 ns, FastSET adaptation time changes little, while E2E time increases; the speedup rises from $1.52\times$ to $4.65\times$. At 900 ns, FastSET reaches the validation target but has test BER 4.39×10^{-4}, compared with 2.94×10^{-5} for E2E.

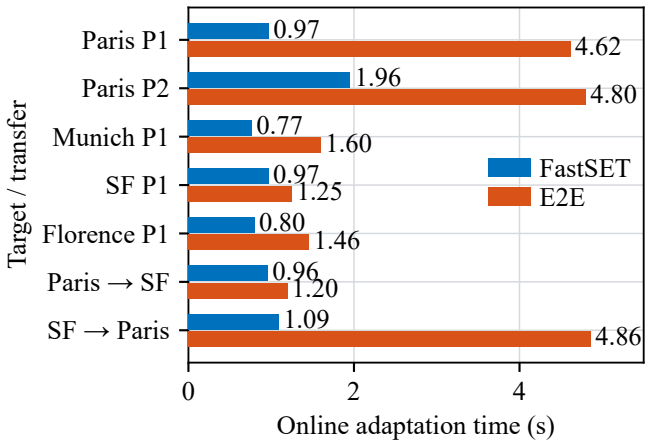


Fig. 19: FastSET and E2E adaptation times for seven RT target conditions. Labels report time in seconds; adaptation stops when validation BER falls below 2×10^{-4} at 12 dB.

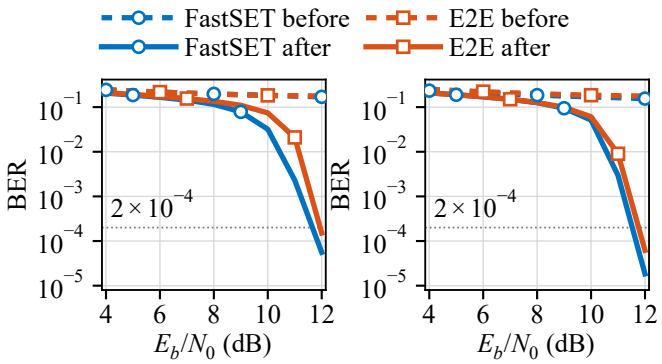


Fig. 20: FastSET and E2E BER before and after adaptation at Paris P1 (left) and P2 (right), using the same source-pretrained receivers. Adapted receiver weights remain fixed across SNRs.

VI. Discussion

Split Selection and Pretraining. Pretraining affects target-domain BER and the iterations required for adaptation. Pretraining settings are fixed to compare splits and update strategies; pretrained FastSET and E2E have comparable source-domain BER (Supplementary Section II). Increasing E2E pretraining from 10 to 30 epochs reduces A2B adaptation from 50 to 40 iterations; A2C requires 84 and 85 iterations, respectively. FastSET retains the time advantage. Target-specific pretraining budgets could also be optimized; fixed settings allow the selected split to be evaluated under a common protocol.

Stopping Criterion. BER-checking frequency affects evaluation cost and stopping time. Per-iteration checks identify the first qualifying model; input processing and training are timed with evaluation disabled. Practical receivers may check periodically or use fixed budgets. Supplementary Section IV estimates the costs: fewer checks reduce evaluation overhead, while delayed stopping retains additional FastSET iterations and narrows the time advantage. Checking intervals can be selected according to evaluation cost and expected adaptation speed. Supplementary Section VII evaluates the selected models on independent samples.

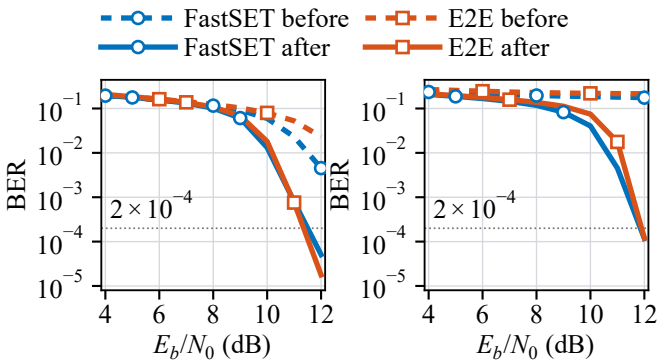


Fig. 21: FastSET and E2E BER before and after cross-city adaptation. Left: Paris source to San Francisco P1. Right: San Francisco source to Paris P1.

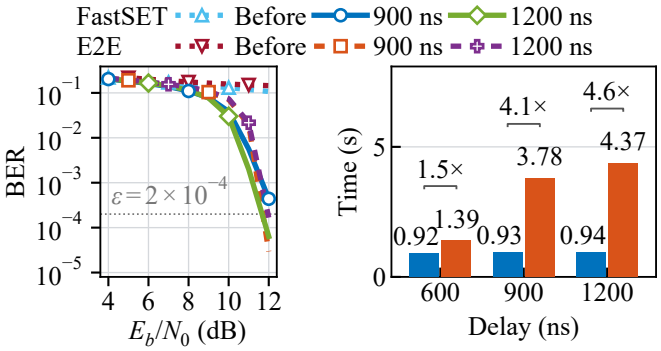


Fig. 22: Delay-spread sensitivity at Paris P1. Left: BER after adaptation at 900 and 1200 ns, with pretrained curves for reference. Right: computation time to the validation BER target at 600, 900, and 1200 ns, with the E2E/FastSET time ratio.

Practical Deployment. Channel-label availability and quality affect adaptation and deployment cost. Simulated channel responses and known transmitted bits provide the estimator and demapper labels, respectively. Future work will construct channel labels from known training symbols in a practical receiver, evaluate adaptation with label estimation errors, and measure label construction, training, and model activation costs.

VII. Conclusion

FastSET uses a physics-guided estimator–demapper split to adapt reception through an explicit channel estimate. Offline PEBS search selects the module architectures under a common parameter budget. Online adaptation updates the estimator first and then the demapper as needed to reach the target BER. The ablations show that estimator adaptation alone can recover the target BER, while stricter reliability requirements benefit from further estimator and demapper adaptation. The selected 8–64 split reduces adaptation computation time by factors of 86.3 on A2B and 22.0 on A2C, with comparable post-adaptation BER to E2E. Evaluation in four RT cities further demonstrates adaptation-time reductions across different propagation environments. By aligning online adaptation with the channel-estimation and demapping functions of conventional receivers, FastSET reduces the computation required to restore reliable

reception under changing propagation conditions. Physics-guided online adaptation can therefore facilitate the deployment of neural receivers and support the broader application of AI in wireless communication systems.

REFERENCES

[1] F. Hu, Y. Li, H. Zhu, and X. Wang, "FastSET: Fast sequential retraining to accelerate real-time online adaptation for neural receivers," in *Proc. IEEE Int. Conf. Sensing, Communication, and Networking (SECON)*, 2026, pp. 150–158.

[2] C.-J. Chun, J.-M. Kang, and I.-M. Kim, "Deep learning-based channel estimation for massive MIMO systems," *IEEE Wireless Communications Letters*, vol. 8, no. 4, pp. 1228–1231, 2019.

[3] E. Balevi, A. Doshi, A. Jalal, A. Dimakis, and J. G. Andrews, "High dimensional channel estimation using deep generative networks," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 1, pp. 18–30, 2021.

[4] D. Luan and J. S. Thompson, "Achieving robust generalization for wireless channel estimation neural networks by designed training data," in *IEEE International Conference on Communications (ICC)*, 2023, pp. 3462–3467.

[5] K. Chahine, N. Ye, and H. Kim, "DeepIC: Coding for interference channels via deep learning," in *IEEE Global Communications Conference (GLOBECOM)*, 2021, pp. 01–06.

[6] M. Yang, C. Bian, and H.-S. Kim, "OFDM-guided deep joint source channel coding for wireless multipath fading channels," *IEEE Transactions on Cognitive Communications and Networking*, vol. 8, no. 2, pp. 584–599, 2022.

[7] S. Park, H. Jang, O. Simeone, and J. Kang, "Learning to demodulate from few pilots via offline and online meta-learning," *IEEE Transactions on Signal Processing*, vol. 69, pp. 226–239, 2021.

[8] K. M. Cohen, S. Park, O. Simeone, and S. Shamai, "Bayesian active meta-learning for reliable and efficient AI-based demodulation," *IEEE Transactions on Signal Processing*, vol. 70, pp. 5366–5380, 2022.

[9] Y. Liang, L. Li, Y. Yi, and L. Liu, "Real-time machine learning for symbol detection in MIMO-OFDM systems," in *IEEE INFOCOM*, 2022, pp. 2068–2077.

[10] O. Wang, J. Gao, and G. Y. Li, "Learn to adapt to new environments from past experience and few pilot blocks," *IEEE Transactions on Cognitive Communications and Networking*, vol. 9, no. 2, pp. 373–385, 2023.

[11] H. Ye, G. Y. Li, and B.-H. Juang, "Power of deep learning for channel estimation and signal detection in OFDM systems," *IEEE Wireless Communications Letters*, vol. 7, no. 1, pp. 114–117, 2018.

[12] M. Honkala, D. Korpi, and J. M. J. Huttunen, "DeepRx: Fully convolutional deep learning receiver," *IEEE Transactions on Wireless Communications*, vol. 20, no. 6, pp. 3925–3940, 2021.

[13] Y. Liao, N. Farsad, N. Shlezinger, Y. C. Eldar, and A. J. Goldsmith, "Deep neural network symbol detection for millimeter wave communications," in *IEEE Global Communications Conference (GLOBECOM)*, 2019, pp. 1–6.

[14] X. Gao, S. Jin, C.-K. Wen, and G. Y. Li, "ComNet: Combination of deep learning and expert knowledge in OFDM receivers," *IEEE Communications Letters*, vol. 22, no. 12, pp. 2627–2630, 2018.

[15] S. Zheng, S. Wu, H. Li, C. Jiang, and X. Jing, "Deep learning-aided receiver against nonlinear distortion of HPA in OFDM systems," in *IEEE International Conference on Wireless Communications and Signal Processing (WCSP)*, 2021, pp. 1–6.

[16] M. Goutay, F. Ait Aoudia, J. Hoydis, and J.-M. Gorce, "Machine learning for MU-MIMO receive processing in OFDM systems," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 8, pp. 2318–2332, 2021.

[17] T. Raviv, S. Park, O. Simeone, Y. C. Eldar, and N. Shlezinger, "Online meta-learning for hybrid model-based deep receivers," *IEEE Transactions on Wireless Communications*, vol. 22, no. 10, pp. 6415–6431, 2023.

[18] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "MnasNet: Platform-aware neural architecture search for mobile," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2820–2828.

[19] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *International Conference on Machine Learning*, vol. 70, 2017, pp. 1126–1135.

[20] N. Uzlaner, T. Raviv, N. Shlezinger, and K. Todros, "Asynchronous online adaptation via modular drift detection for deep receivers," *IEEE Transactions on Wireless Communications*, vol. 24, no. 5, pp. 4454–4468, 2025.

[21] S. Wang, T. Wang, and X. Wang, "FedPDA: Collaborative learning for reducing online-adaptation frequency of neural receivers," in *IEEE INFOCOM*, 2025, pp. 1–10.

[22] J. Hoydis, S. Cammerer, F. Ait Aoudia, A. Vem, N. Binder, G. Marcus, and A. Keller, "Sionna: An open-source library for next-generation physical layer research," 2022, arXiv preprint arXiv:2203.11854.

[23] J. Hoydis, F. Ait Aoudia, S. Cammerer, M. Nimier-David, N. Binder, G. Marcus, and A. Keller, "Sionna RT: Differentiable ray tracing for radio propagation modeling," 2023, arXiv preprint arXiv:2303.11103.



Shuo Wang received the bachelor's degree in electronic engineering from Tsinghua University, Beijing, China, and the master's degree in computer science from Shanghai Jiao Tong University, Shanghai, China. He is currently a search algorithm engineer with Alibaba Group. His research interests include machine learning and cloud computing.



Facheng Hu received his bachelor's and master's degrees from Shanghai Jiao Tong University, Shanghai, China, where he is currently pursuing a Ph.D. under the supervision of Prof. Yibo Pi. His research interests include intelligent networking and sensing.



Hongzi Zhu received the PhD degree in computer science from Shanghai Jiao Tong University, in 2009. He was a post-doctoral fellow with the Department of Computer Science and Engineering, Hong Kong University of Science and Technology, and the Department of Electrical and Computer Engineering, University of Waterloo, in 2009 and 2010, respectively. He is now a professor with the School of Computer Science, Shanghai Jiao Tong University. His research interests include mobile sensing, mobile computing, and Internet of Things. He received the Best Paper Award from IEEE Globecom 2016. He is an associate editor for the IEEE Transactions on Vehicular Technology and the IEEE Internet of Things Journal. He is an IEEE Senior Member. For more information, please visit http://lion.sjtu.edu.cn.



Yibo Pi (Member, IEEE) earned his B.S. and M.S. from Shanghai Jiao Tong University, Shanghai, China, in 2012 and 2015, respectively, and completed his Ph.D. in computer science and engineering at the University of Michigan, Ann Arbor, MI, USA, in 2021. He is an associate professor at the Global College of Shanghai Jiao Tong University. His research focuses on wireless sensing, wireless networking, and industrial Internet of Things systems.



Xudong Wang (Fellow, IEEE) completed his Ph.D. in electrical and computer engineering at the Georgia Institute of Technology, Atlanta, GA, USA, in 2003. He is a professor in the Internet of Things Thrust at The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China, and an affiliate professor at the University of Washington. His research covers wireless communication networks, distributed machine learning, mobile edge computing, and joint communications and sensing.